

Table of Contents

RAKTA ADS — Autoware × IPG CarMaker Integration

Technical Handover & Rebuild Specification (SP1)

Document owner: Ali Alsaïd — Systems Integration Engineer, EEE (AURAK) **Sponsor:** RAKTA Autonomous Office · Supervisor: Dr. Vima Rau **Status at handover:** Forward control loop closed and verified (Level 1). Return-feedback path (Level 2) designed, in implementation. **Purpose:** Enable the RAKTA engineering team to rebuild the complete integration from a bare machine with no author present. Every path, version, variable, and command is explicit.

A. Visual Documentation & Design Rationale

This part was added to make the document teachable, not just referenceable. Every diagram is built **only** from data verified in the development record (git commit 6182620, the bridge source, and the working session logs). Diagrams describing the **unbuilt** return path are explicitly marked. Screenshots that can only come from the live machine are inserted as labeled placeholders — they must be captured by the operator before final handover and **must not be faked**.

A.1 What Each Component Does (Responsibility Matrix)

A newcomer's first question is "who owns what." This table answers it before any command is run.

Component	Layer	Responsibility	Language / Tech
Autoware Universe	Brain	Localization, perception, path planning, and control. Decides <i>what the vehicle should do</i> (target acceleration + steering).	C++/Python, ROS 2
bridge_node.py	Translator	Converts Autoware's control message (m/s ² , rad) into CarMaker's actuator message (gas/brake 0–1, steer rad). Owns rate adaptation (ZOH). The heart of SP1.	Python / rclpy

Component	Layer	Responsibility	Language / Tech
CMNode (CMRosIF)	In-sim ROS node	A ROS 2 node running <i>inside</i> the CarMaker process. Receives <code>VehicleControl</code> , applies it to the vehicle; (Level 2) will read vehicle state and publish it back.	C++
IPG CarMaker 15.0	Physics	Simulates vehicle dynamics, road, environment, and sensors. Executes the physical response to actuator commands.	IPG proprietary
IPGControl	Debug	Live plotting of internal CarMaker quantities (<code>VC.Gas</code> , <code>Car.v</code> , DVA signals). Used to confirm physical obedience.	IPG tool
RViz	Visualization	Renders Autoware's world view: pose, planned path, map. Used to set pose/goal and engage Auto.	ROS 2 tool
CycloneDDS	Transport	The DDS middleware that carries every ROS 2 topic between processes.	ROS 2 RMW

A.2 Design Rationale — Why It Is Built This Way

The review correctly noted the document explained *how* but not always *why*. These are the load-bearing decisions.

Why a separate bridge node, instead of modifying Autoware or CarMaker directly?

Three reasons. (1) **Separation of concerns** — Autoware and CarMaker are large third-party codebases; editing either forks it and makes every future upstream update painful. The bridge is a small, owned, replaceable adapter. (2) **Unit/interface mismatch is a translation problem**, and a translator is the natural place to solve it: Autoware speaks physical units (m/s^2 , rad), CarMaker speaks normalized actuator levels (0–1). (3) **Testability** — the bridge can be unit-tested and echo'd in isolation; a patch buried inside Autoware cannot.

Why Python for the bridge (not C++)? Confirmed by Dr. Vima Rau. The bridge is light arithmetic at modest rates; `rcipy` is sufficient and far faster to write, read, and hand over. C++ was reserved for the one place it is unavoidable — the in-process CMNode, which must link against CarMaker's C/C++ API.

Why is the return path C++ (CMNode) and not Python? The return path must read CarMaker's internal physics state **tick-synchronized** with the simulation. That state lives in CarMaker's process memory and is only reachable through its C/C++ DVA API, from inside the CarMaker binary. A Python node outside the process cannot see tick-accurate state. So the read must be C++ in CMNode; only the downstream unit conversion returns to Python in the bridge.

Why 10 Hz in and 1000 Hz out (ZOH)? Autoware’s controller publishes control commands at ~10 Hz — that is the rate at which new decisions are made. CarMaker’s solver runs its vehicle model far faster and expects a command every step. If the bridge only published at 10 Hz, CarMaker would see stale/empty input between updates. The **Zero-Order Hold** republishes the last decision at 1000 Hz, holding it steady until the next one arrives — exactly like a sample-and-hold holding a voltage between clock edges. (EEE analogy retained from the working notes.)

Why dlopen at runtime instead of statically linking CMNode into CarMaker? CMRosIF is distributed as a prebuilt shared library. Runtime loading lets the ROS node be rebuilt and swapped without recompiling the whole CarMaker binary, and keeps IPG’s binary and the ROS interface as separate build units. The cost is the -Wl, -E requirement (export CarMaker’s symbols so the loaded .so can resolve them) — documented in §6.8.

Why use_vc=True and selector_ctrl=1? use_vc switches CarMaker from its built-in IPGDriver to external VehicleControl input — without it, Autoware’s commands are ignored and the car drives itself. selector_ctrl=1 puts the transmission in Drive. Both were empirically required (bug #6 in §5/Phase-4 record).

A.3 Engineering Assumptions (Explicit)

The system is valid **only** under these assumptions. An engineer changing any of them must re-verify.

#	Assumption	Where it matters
1	Autoware publishes control_cmd at ~10 Hz	Bridge ZOH design; if Autoware rate changes, revisit timer logic
2	Bridge timer runs at 1000 Hz	Output rate to CarMaker
3	Steering is in radians end-to-end (Autoware steering_tire_angle → CarMaker steer_ang)	Direct pass-through with no unit conversion
4	Velocity is in m/s (CarMaker Car.v)	Return path conversion to VelocityReport
5	MAX_ACCEL = MAX_DECEL = 3.0 m/s ² is the normalization basis	Gas/brake scaling; vehicle-specific — confirm against the actual vehicle’s limits
6	ROS 2 Humble + CycloneDDS only	Message definitions and RMW behavior
7	Single ego vehicle, right-hand coordinate convention	Map alignment, pose injection
8	CarMaker custom binary (bin/CarMaker.linux64) is the one launched	Entire forward loop; see §3.5

A.4 Risk Register

Risk	Likelihood	Impact	Mitigation (documented)
CarMaker GUI silently overwrites the TestRun	High (every GUI open)	Sim won't run	chmod 444 the TestRun (§3.5, §6.9)
Wrong binary launched (stock instead of custom)	Medium	Subscription count: 0, silent	Verify CM.Exe = CarMaker.linux64 in Data/Config/GUI (§3.5)
CMRosIFParameters missing/empty	Medium	Loader aborts silently	Ship the file with Cfg.Mode=1 + abs .so path (§6.11)
CarMaker license expires (2026-07-31) or machine changes	Certain (dated)	CarMaker won't start	Renew via IPG; node-locked to MAC+hostname (§4 / Troubleshooting #2)
Map origin mismatch (Autoware vs CarMaker)	High at integration	Car drives off track	Shared georeferenced origin + static TF (§2.3) — open
Git repo / zip host goes down	Medium	Cannot rebuild bridge or project	Mirror to RAKTA-owned storage; §6.12 documents files to rebuild from scratch
GPU sensors crash on hybrid GPU	High on this HW	Sim stalls at init	Disable Sensors 4/5/6 (§3.5, §6.7)

A.5 Known-Good Version Lock (Reproducibility Baseline)

Record the exact configuration that produced the verified Level 1 result. Builds months later should pin to these.

Item	Known-good value
OS	Ubuntu 22.04 LTS
ROS 2	Humble
RMW	CycloneDDS (rmw_cyclonedds_cpp)
IPG	15.0
CarMaker	
CUDA	12.8
NVIDIA driver	580.x
Autoware	482 packages, <FILL-IN: record the autoware commit hash – run \cd ~/autoware && git rev-parse HEAD`>     Bridge   <FILL-IN: bridge_node commit hash>     CarMaker project fix   commit6182620` (verified)
CMRosIF	shipped with CarMaker 15.0 add-on

Action: fill the three <FILL-IN> commit hashes from the respective repos before handover. Commit 6182620 is already known and verified.

A.6 Figures (built from verified data)

The following figures are embedded after this section in the Word export. Each caption states its data source.

- **Figure 1 — System Data Flow (Closed Loop).** The full forward (verified) and return (pending) topic path. Source: §2.
- **Figure 2 — Bridge Node Internal Architecture.** The two callbacks, shared state, and conversion. Source: §6.10 verified code.
- **Figure 3 — CMRosIF Binary & Loader Chain.** Why the custom binary is mandatory; failure modes. Source: git commit 6182620.
- **Figure 4 — Repository & Filesystem Structure.** The three isolated trees. Source: §1.2, §3.3.
- **Figure 5 — Boot-to-Drive Lifecycle.** The strict launch order with dependency rationale. Source: §3.6.
- **Figure 6 — Message Conversion Worked Example.** Real values (accel 1.8 → gas 0.60) through the verified math. Source: §6.10.
- **Figure 7 — Bridge Package Dependency Graph.** Verified imports + package.xml. Source: §6.12.

Figure 1 — System Data Flow (Closed Loop)
Forward path = solid navy (Level 1, verified) - Return path = dashed gold (Level 2, In Implementation)

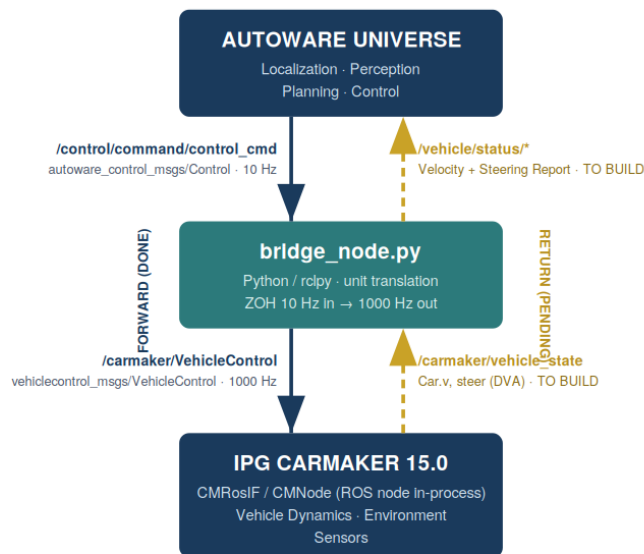


Figure 1 — System Data Flow

Figure 2 — Bridge Node Internal Architecture

Two callbacks, decoupled by shared state. Source: §6.10 (verified bridge_node.py)

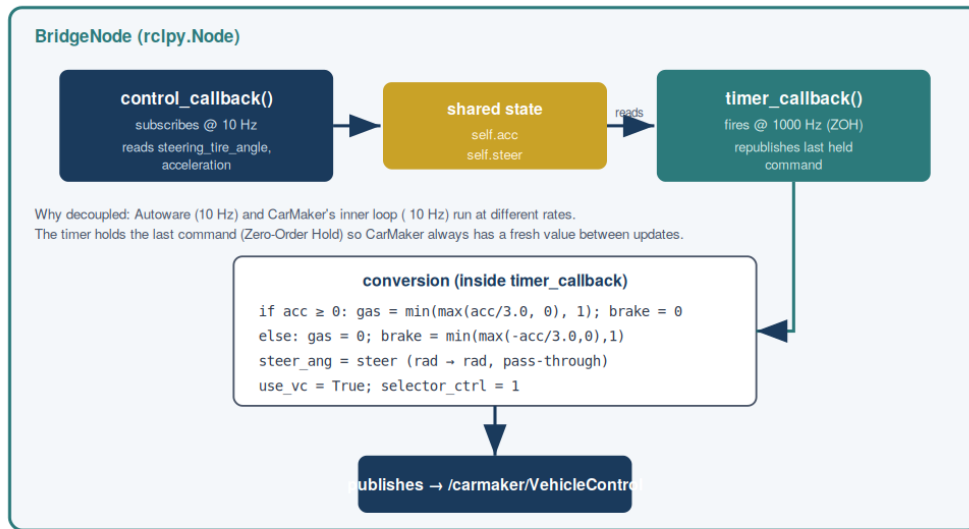


Figure 2 — Bridge Node Internal Architecture

Figure 3 — CMRosIF Binary & Loader Chain

Why the custom binary is mandatory. Source: git commit 6182620 (verified diff)

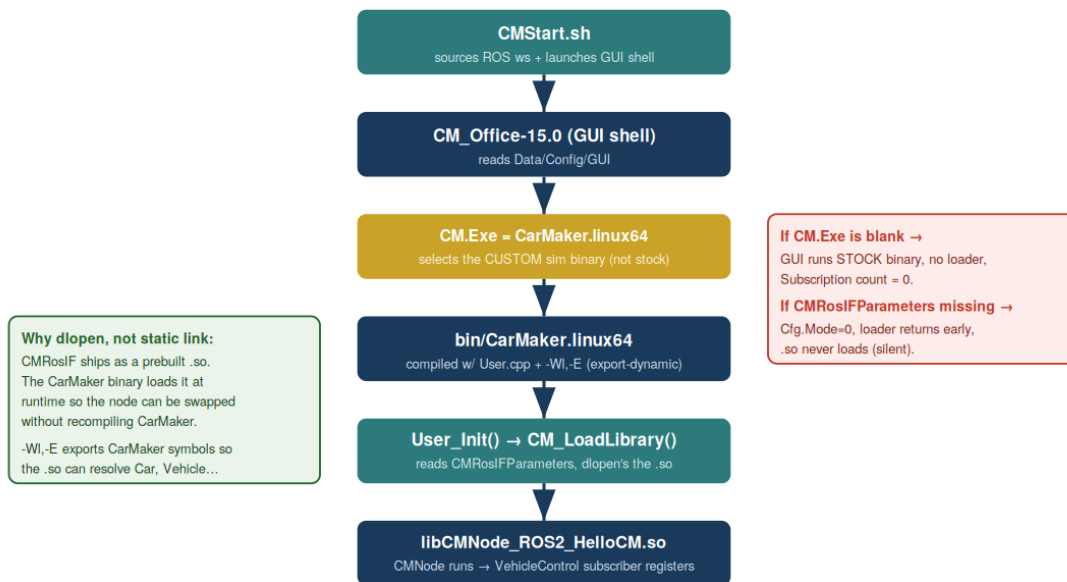


Figure 3 — CMRosIF Binary and Loader Chain

Figure 4 — Repository & Filesystem Structure

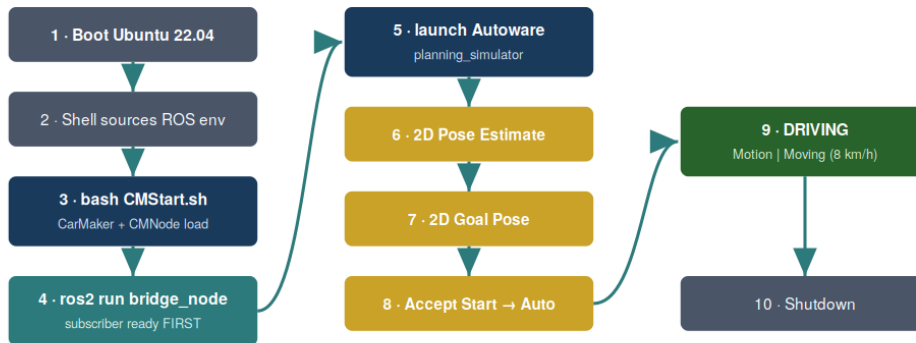
Three isolated trees. Source: §1.2, §3.3 (verified paths)



Figure 4 — Repository and Filesystem Structure

Figure 5 — Boot-to-Drive Lifecycle

Strict order. Source: §3.6 (verified launch sequence)



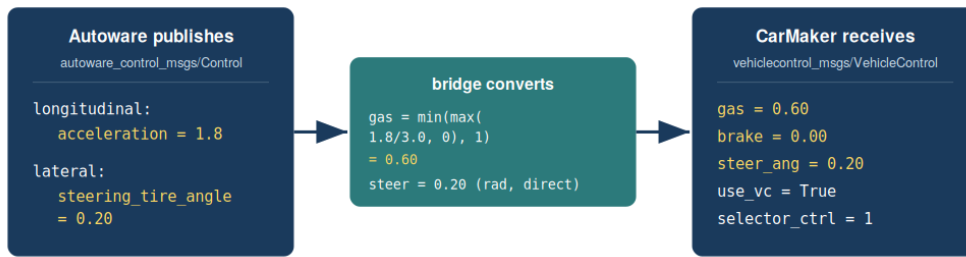
Dependency rationale: CarMaker first (CMNode must exist) → bridge second (subscriber must be ready) → Autoware last (publisher begins only when a listener exists). Steps 6–8 are GUI actions in RViz.

Note: gold steps (6–8) require the map-origin alignment of §2.3 before Mode B closed-loop tracking is valid.

Figure 5 — Boot-to-Drive Lifecycle

Figure 6 — Message Conversion: Worked Example

Real values through the verified conversion (MAX_ACCEL = 3.0). Source: §6.10



Verification: run `ros2 topic echo /carmaker/VehicleControl` while Autoware commands 1.8 m/s² — you should see gas ≈ 0.60.

A positive acceleration maps to gas only (brake = 0); negative would map to brake only (gas = 0).

Figure 6 — Message Conversion Worked Example

Figure 7 — Bridge Package Dependency Graph

Source: verified Imports in `bridge_node.py` + `package.xml` (§6.12)

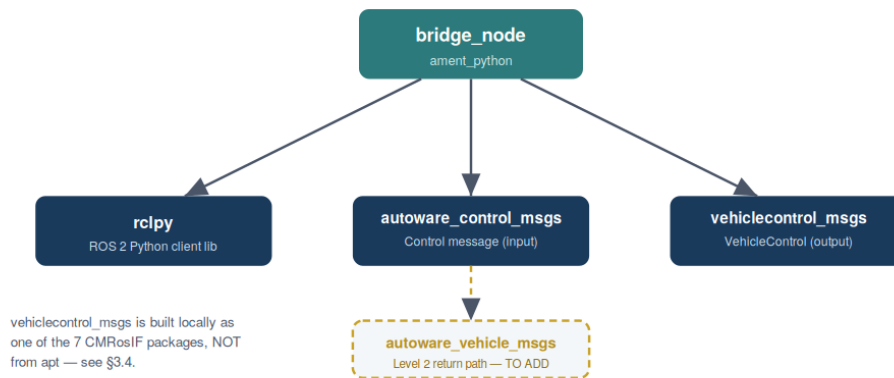


Figure 7 — Bridge Package Dependency Graph

A.7 Required Screenshots — TO BE CAPTURED (placeholders, do not fake)

These cannot be generated; they must be captured from the running system by the operator and pasted in before handover. Each box states exactly what the image must show so its evidentiary value is unambiguous.

[SCREENSHOT 1 — Subscription count: 1] Terminal running `ros2 topic info /carmaker/VehicleControl --verbose`, showing the line `Subscription count: 1`. This is the single most important proof that the forward loop is connected. Capture with CarMaker running the custom binary.

[SCREENSHOT 2 — CMNode alive] The `CMStart.sh` terminal showing the startup print `hellocm: I am Alive` (and any CMRosIF “online” status line). Proves the `.so` loaded.

[**SCREENSHOT 3 — RViz driving**] RViz with the vehicle on the map, a planned trajectory drawn, and the OperationMode panel showing AUTONOMOUS / Motion: Moving. Proves Autoware engaged. Annotate the speed if visible (~8 km/h).

[**SCREENSHOT 4 — IPGControl physical obedience**] IPGControl plotting VC.Gas and Car.v rising together while Autoware commands forward motion. Proves the vehicle physically obeys, not just that the topic is subscribed.

[**SCREENSHOT 5 — ROS graph (rqt_graph)**] rqt_graph showing the node/topic topology: Autoware → /control/command/control_cmd → bridge_node → /carmaker/VehicleControl → CMNode. Gives a newcomer the live wiring at a glance.

[**SCREENSHOT 6 — Topic list**] Output of `ros2 topic list` with the relevant control and (once built) /vehicle/status/* topics visible. Quick sanity reference.

A.8 Return-Path Sequence — PLACEHOLDER (Level 2 not yet built)

[**DIAGRAM — to be drawn once the return path exists**] A sequence diagram of the **closed** loop: Autoware publishes Control → bridge converts → CMNode applies → CarMaker steps physics → CMNode reads Car.v + steering DVA → publishes /carmaker/vehicle_state → bridge converts → VelocityReport/SteeringReport → Autoware localizer/controller closes the loop.

This is intentionally **not** drawn yet: the exact steering DVA name is unconfirmed (§7.1) and the publisher does not exist (§7.2). Drawing it now would imply built functionality that is not. It will be added — with real topic names and the confirmed DVA — once §7 is implemented and verified against the §7.4 thresholds.

1. System Requirements Specification (Understand)

1.1 Objective

Connect **Autoware Universe** (open-source autonomous driving stack) to **IPG CarMaker 15.0** (vehicle physics simulator) through a custom ROS 2 bridge, so the simulated vehicle is driven by Autoware's planning and control commands instead of CarMaker's built-in IPGDriver.

The integration is a closed control loop with two halves:

- **Forward path** — Autoware control commands → bridge → CarMaker actuators. **(Complete.)**
- **Return path** — CarMaker vehicle state (speed, steering) → bridge → Autoware vehicle-status inputs. **(In implementation.)**

End goal of the wider program: reproduce the Al Hamra field driving anomaly in simulation on the Al Marjan HD map.

1.2 Hardware & OS Baseline (Strict)

Requirement	Specification	Reason
OS	Ubuntu 22.04 LTS	Autoware Universe + ROS 2 Humble target 22.04. Do not use 24.04.
Install type	Native dual-boot	WSL2 cannot run CMRosIF and cannot use the GPU reliably.
RAM	≥ 24 GB	Autoware colcon build exhausts memory below this; WSL2's RAM cap caused OOM kills.
GPU	NVIDIA (e.g. T500) + driver 580.x, CUDA 12.8	Required for RViz and perception. Reference machine is an Intel/NVIDIA PRIME hybrid (HP ZBook Firefly 15 G8).
Disk for Linux	≥ 175 GB	Autoware build tree, CarMaker, and maps are large.
Boot mode	UEFI	Required for the dual-boot partition scheme below.

Pre-install warning (critical): If Windows BitLocker is enabled, **suspend it before repartitioning** or the machine can lock and require the recovery key. This is the most common dual-boot failure.

Reference partition scheme (manual “Something else” install):

Partition	Mount	Size	Note
nvme0n1p1	EFI	existing	Reuse, do not format
nvme0n1p5	swap	16 GB	—
nvme0n1p6	ext4 /	171 GB	—

1.3 Software Dependencies

Component	Version
ROS 2	Humble
RMW (middleware)	CycloneDDS (rmw_cyclonedds_cpp)
Autoware	Universe (built: 482 packages)
IPG CarMaker	15.0 (license-bound — see §4)
CarMaker ROS interface	CMRosIF (Linux-only)
Bridge language	Python 3 / rclpy (confirmed by Dr. Vima Rau; C++ not required for the bridge node)
Return-path language	C++ (CMNode DVA, in-memory)

Required apt packages (ROS 2 base):

```
sudo apt install software-properties-common -y
sudo apt install curl -y
sudo apt install ros-humble-desktop-full -y
sudo apt install python3-colcon-common-extensions python3-rosdep -y
sudo apt install ros-humble-rmw-cyclonedds-cpp -y
```

1.4 Environment Variables & ~/.bashrc Additions

CMRosIF builds fail without CARMAKER_VER and IPGHOME. These must be present in **every shell that builds CMRosIF**. Add the block below to ~/.bashrc:

```
# --- ROS 2 ---
source /opt/ros/humble/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# --- Autoware ---
source ~/autoware/install/setup.bash

# --- Main ROS workspace (bridge) ---
source ~/ros2_ws/install/setup.bash

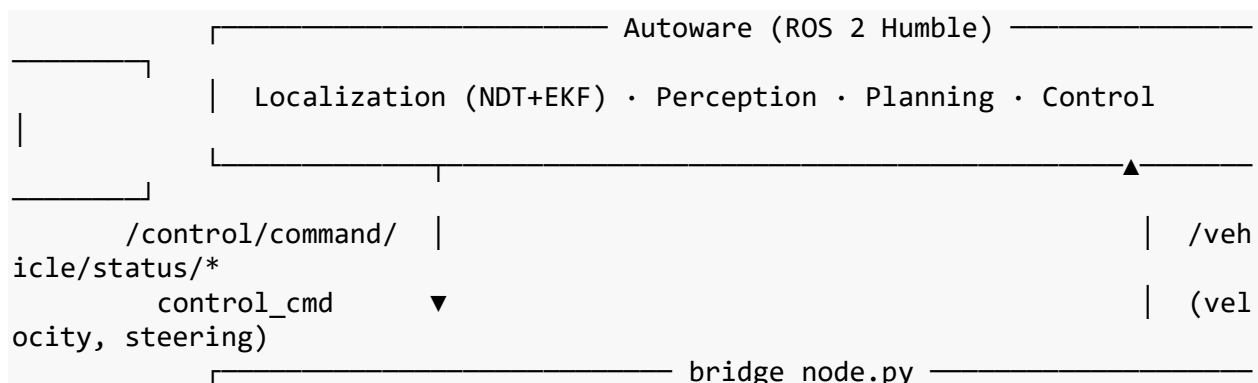
# --- IPG CarMaker / CMRosIF build env ---
export CARMAKER_VER=15.0
export IPGHOME=/opt/ipg
```

On a freshly installed OS, the source lines will error (No such file or directory) until each component is installed. This is expected; the errors disappear as ROS 2, Autoware, and the workspace are built.

2. System Architecture & Data Flow (Organize)

2.1 High-Level Logic

The system is a closed loop running across two processes joined by ROS 2 topics.




```
if accel >= 0:
    gas    = min(accel / MAX_ACCEL, 1.0)    # clamp: gas > 1.0 faults the sim
    brake  = 0.0
else:
    gas    = 0.0
    brake  = min(-accel / MAX_DECEL, 1.0)

steer_ang    = steer    # radians → radians (direct pass-through)
use_vc       = True     # enable external control, override IPGDriver
selector_ctrl = 1       # Drive
```

Return path — CarMaker → Autoware (in implementation):

Direction	Topic	Message Type	Source
CMNode publishes	/carmaker/<vehicle_state> (raw)	TBD (e.g. Float64MultiArray)	DVA reads Car.v, Car.Steer.Ang each tick
bridge subscribes	/carmaker/<vehicle_state>	(matches above)	—
bridge publishes	/vehicle/status/velocity_status	autoware_vehicle_msgs/msg/VelocityReport	from Car.v
bridge publishes	/vehicle/status/steering_status	autoware_vehicle_msgs/msg/SteeringReport	from Car.Steer.Ang
bridge publishes	/vehicle/status/control_mode	autoware_vehicle_msgs/msg/ControlModeReport	mode = AUTONOMOUS

Why the return path matters: Without it, Autoware drives blind on its own internal kinematic model. The /carmaker/cm2ext message carries only header, cycleno, synthdelay — no vehicle state. Real physics feedback is required to close the loop (Level 2).

See Figure 1 (System Data Flow) in Part A for the visual of this interface — forward path solid, return path dashed.

Per-Topic Interface Specification (formal)

INTERFACE 1 — Control command (Autoware → bridge), VERIFIED | Field | Value | |—|—| | Topic | /control/command/control_cmd | | Message | autoware_control_msgs/msg/Control | | Publisher | Autoware (control module) | |

Subscriber | bridge_node | | Rate | ~10 Hz | | Used fields | lateral.steering_tire_angle (rad), longitudinal.acceleration (m/s²) | | QoS | default (reliable) |

INTERFACE 2 — Vehicle control (bridge → CarMaker/CMNode), VERIFIED | Field | Value | |—|—| | Topic | /carmaker/VehicleControl | | Message | vehiclecontrol_msgs/msg/VehicleControl | | Publisher | bridge_node | | Subscriber | CMNode (in CarMaker) | | Rate | 1000 Hz (ZOH) | | Set fields | gas [0–1], brake [0–1], steer_ang (rad), steer_ang_vel=0, steer_ang_acc=0, selector_ctrl=1, use_vc=True | | Verify | ros2 topic info /carmaker/VehicleControl --verbose → Subscription count: 1 |

INTERFACE 3 — Vehicle state (CarMaker → bridge), TO BUILD (Level 2) | Field | Value | |—|—| | Topic | /carmaker/vehicle_state (name TBD) | | Message | TBD (candidate std_msgs/Float64MultiArray or a custom vehiclestate_msgs) | | Publisher | CMNode (new — §7.2) | | Subscriber | bridge_node (new — §7.3) | | Source | DVA reads: Car.v, steering DVA (name unconfirmed — §7.1) | | Status | **not implemented** |

INTERFACE 4 — Vehicle status (bridge → Autoware), TO BUILD (Level 2) | Field | Value | |—|—| | Topics | /vehicle/status/velocity_status, /vehicle/status/steering_status, /vehicle/status/control_mode | | Messages | autoware_vehicle_msgs/msg/{VelocityReport, SteeringReport, ControlModeReport} | | Publisher | bridge_node (new) | | Subscriber | Autoware (EKF localizer, controller) | | Status | **not implemented** |

2.3 Map Coordinate Alignment (OPEN CONSTRAINT — must be resolved before Al Marjan)

 **This is an unsolved engineering problem, not a documentation gap. Read before any closed-loop map test.**

Autoware and CarMaker each load their **own, independent** map with its **own origin**:

Stack	Map (current)	Origin frame
Autoware	nishishinjuku_autoware_map (Lanelet2 + PCD)	Autoware map frame origin (0,0,0)
CarMaker	FS_autonomous_TrackDrive.rd5	CarMaker Fr0 inertial origin

During Level 1 these two were **never aligned** — the test proved Autoware’s planning/control and the forward bridge in isolation (car moved at 8 km/h driven by Autoware’s own kinematic model), **not** that Autoware was localizing against the same world CarMaker simulates. They are presently two different maps with two different origins.

The constraint: for a true closed-loop run, Autoware’s map origin and CarMaker’s Fr0 must describe the **same physical world**, or the vehicle Autoware *thinks* it is driving will diverge from the vehicle CarMaker *physically* drives, and the car leaves the track.

How it must be resolved (the plan, not yet implemented): 1. Both stacks must load the **same** geographic area — Mahra's Al Marjan map, delivered as a paired **Lanelet2 .osm (for Autoware)** and the matching **CarMaker road (.rd5)** built from the same source geometry. 2. Define a single **georeferenced origin** (a fixed lat/lon/elevation reference point) used by both: Autoware's Lanelet2 MGRS/UTM projection origin and CarMaker's GCSProjection.RefPoint.LLE must be set to the **same** reference. (Note: the FS_autonomous_TrackDrive.rd5 already carries a GCSProjection.RefPoint.LLE field — see the road file — confirming CarMaker supports a georeferenced origin; it simply isn't matched to Autoware's yet.) 3. Publish a static TF (map → CarMaker Fr0) capturing any residual offset/rotation, validated with `ros2 run tf2_ros tf2_echo map base_link` against CarMaker's reported pose once the return path exists.

Acceptance test for alignment: with the return path live, command Autoware to hold position; CarMaker's reported Car.tx/ty (in Fr0) transformed through the static TF must match Autoware's base_link in map to within the tracking margin defined in §8. If they drift apart, the origins are not aligned.

3. Execution Roadmap (Simplify)

Sequential. Each step states the why, then the exact commands.

3.1 Install ROS 2 Humble

Why: Base middleware for both Autoware and the bridge.

```
# Enable universe + add ROS 2 apt source
sudo apt install software-properties-common -y
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# Install ROS 2 + build tools + CycloneDDS
sudo apt update && sudo apt upgrade -y
sudo apt install ros-humble-desktop-full -y
sudo apt install python3-colcon-common-extensions python3-rosdep -y
sudo rosdep init && rosdep update
sudo apt install ros-humble-rmw-cyclonedds-cpp -y

# Verify (note: `ros2 --version` is NOT valid)
```

```
source /opt/ros/humble/setup.bash
printenv ROS_DISTRO          # expect: humble
```

3.2 Install & Build Autoware Universe

Why: The planning/control brain. The setup script handles CUDA/cuDNN/TensorRT.

```
# Clone
cd ~
git clone https://github.com/autowarefoundation/autoware.git
cd autoware

# Setup dev environment (answer yes to NVIDIA + CUDA)
./setup-dev-env.sh
```

KNOWN ISSUE (apt source conflict): If the manual ROS 2 source from §3.1 conflicts with the one the ansible script adds, setup-dev-env.sh aborts. Fix and re-run:

```
sudo rm /etc/apt/sources.list.d/ros2.list
sudo apt update
cd ~/autoware && ./setup-dev-env.sh

# Import source repositories
# NOTE: the repos file lives under repositories/, not the autoware root.
vcs import src < repositories/autoware.repos

# Resolve dependencies
source /opt/ros/humble/setup.bash
rosdep install -y --from-paths src --ignore-src --rosdistro humble

# Build (60-90 min; CPU at 100%; do not suspend)
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
```

KNOWN ISSUE (path gotcha): vcs import src < autoware.repos fails with No such file or directory because the file is at repositories/autoware.repos. Use the path shown above. Confirm with find ~/autoware -name "*.repos".

```
# Verify
source ~/autoware/install/setup.bash
ros2 pkg list | grep autoware | wc -l    # expect 300+
echo 'source ~/autoware/install/setup.bash' >> ~/.bashrc
```

Build warnings on ~190 packages are normal (deprecated headers, CMake notes). Only X packages failed in the summary indicates real failure. Reference build: **482 packages, 0 failures, ~2h 21min.**

3.3 Workspace Setup — Isolate CarMaker and ROS Workspaces

Why: Three separate trees must not be cross-contaminated.

⚠ ACTION REQUIRED BEFORE HANDOVER — fill these in. The build cannot start without persistent source locations. Replace each <FILL-IN> with a durable link (RAKTA Git server, SharePoint, or network drive — **not** a personal laptop path or a single supervisor’s local copy). Anything left pointing at a personal location is lost the day that person leaves.

Artifact	Source location	Notes
bridge_node ROS 2 package	<FILL-IN: git clone URL> — dev mirror: github.com/AliaIsaid/bridge_node	Contains bridge_node.py (\$6.10). Move to a RAKTA-owned repo before handover.
Full bridge workspace backup	<FILL-IN: git clone URL> — dev mirror: github.com/AliaIsaid/ros2_ws_backup	Snapshot of ~/ros2_ws.
FS_autonomous CarMaker project	<FILL-IN: SharePoint / network-drive link to FCM_Projects.zip>	From Dr. Vima Rau / IPG. Single point of failure — archive on a shared drive.
CarMaker fix (commit 6182620)	the project’s own git repo at ~/CM_Projects/FS_autonomous/.git	Push to <FILL-IN: RAKTA Git server>. This commit is non- reproducible from any other source.

Tree	Path	Contents
Autoware	~/autoware	The stack (above)
Bridge workspace	~/ros2_ws	bridge_node package
CarMaker project	~/CM_Projects/FS_autonomous (case- sensitive)	CarMaker project + CMRosIF
CMRosIF ROS workspace	~/CM_Projects/FS_autonomous/ros/ros2_ws	The 7 CMRosIF packages

```
# Bridge workspace
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
git clone <FILL-IN: bridge_node repo URL> # places bridge_node package u
nder src/
cd ~/ros2_ws && colcon build --symlink-install
source ~/ros2_ws/install/setup.bash

# CarMaker project
mkdir -p ~/CM_Projects && cd ~/CM_Projects
# Option A (preferred) – clone the fixed project repo, which already contains
```

```
commit 6182620:
git clone <FILL-IN: RAKTA CarMaker project repo URL> FS_autonomous
# Option B – extract the original zip, then apply the fix commit manually:
# unzip FCM_Projects.zip && mv FCM_Projects/FS_autonomous ~/CM_Projects/FS_
autonomous
```

3.4 Compile CMRosIF

Why: CMRosIF provides the in-process ROS node (CMNode) and the custom CarMaker binary that loads it. **CMRosIF is Linux-only.**

The CMRosIF workspace contains **7 packages**: cmrosutils, hellocm_msgs, hellocm, hellocm_cmnode, camera_msgs, vehiclecontrol_msgs, carmaker_rsd_client.

```
export CARMAKER_VER=15.0
export IPGHOME=/opt/ipg
cd ~/CM_Projects/FS_autonomous/ros/ros2_ws

# Clear stale CMake caches before building
rm -rf build install log

# Build the 7 packages
colcon build
```

KNOWN ISSUE (version mismatch in CMake): IPG's scripts ship with a hardcoded legacy version and a missing env value. Before building, apply: -
cmrosutils/CMakeLists.txt: change hardcoded 14.1.1 → 15.0 -
carmaker_rsd_client/CMakeLists.txt: add the missing
"\$ENV{CARMAKER_VER}" value Then wipe caches (rm -rf build install log)
and rebuild. Result: all 7 packages compile.

Build the custom CarMaker binary (the loader)

Why: In IPG's design, libCMNode_ROS2_HelloCM.so is **not auto-loaded**. A loader inside the CarMaker binary must dlopen it. That loader lives in src/User.cpp. A stock /opt/ipg binary has no loader and ignores CMRosIF entirely.

```
# From the CarMaker project source dir containing the Makefile + src/User.cpp
cd ~/CM_Projects/FS_autonomous/src
make # builds bin/CarMaker.Linux64 with the CMRosIF Loader
ls -la ../bin/CarMaker.Linux64 # confirm the custom binary exists
```

MANDATORY edits before this build (root cause of the multi-day blocker): 1. src/User.cpp must contain the **CMRosIF loader code** that dlopens the CMNode .so. If User.cpp is empty stubs, the node never loads and the subscriber never registers. 2. The link step must include the **-Wl, -E (export-dynamic)** flag, so the loaded .so can resolve CarMaker symbols (Car, Vehicle, DrivMan, ...) from the main binary. Expected output: a custom bin/CarMaker.Linux64 (not the stock /opt/ipg binary).

3.5 Link the Custom Binary — Two-Stage Mechanism (CRITICAL)

Why: This is the single most misunderstood part of the build. There are **two** executables, and confusing them re-breaks the Subscription count: 0 fix.

⚠ READ THIS FIRST. CMStart.sh does NOT launch the simulation binary. It launches the **GUI shell** CM_Office-15.0. The GUI then launches the actual **simulation binary** named by the CM.Exe field in Data/Config/GUI. The custom CMRosIF-enabled binary is selected by CM.Exe, never by editing CMStart.sh to point at it directly.

The chain (verified from commit 6182620):

```
CMStart.sh → CM_Office-15.0 (GUI shell) —reads Data/Config/GUI→ CM.Exe =
CarMaker.linux64
                                     |
                                     loads ./bin/CarMaker.linux64
4 (your custom binary)
                                     |
                                     User_Init() dlopen's libCMN
ode_ROS2_HelloCM.so
```

Two files must be set correctly (both done by commit 6182620):

1. Data/Config/GUI — selects the custom simulation binary:

```
CM.Exe = CarMaker.linux64
CM.Exe.History:
    CarMaker.linux64
```

This is what makes the GUI run your compiled bin/CarMaker.linux64 instead of the stock simulation core. If CM.Exe is blank, the GUI runs its default binary, which has **no CMRosIF loader** → Subscription count: 0 returns.

2. CMStart.sh — launches the GUI shell and sources the ROS workspace:

```
#!/bin/bash
source /home/ali/CM_Projects/FS_autonomous/ros/ros2_ws/install/setup.bash
/opt/ipg/bin/CM_Office-15.0 . -apphost localhost -ext GUI/CMExt-CMRosIF.mod -
debuggpu
```

The /opt/ipg/bin/ prefix on CM_Office-15.0 only locates the **GUI launcher** on PATH — it does **not** point at the stock simulation binary. The simulation binary is chosen by CM.Exe above. (The earlier sed in §6.4 that added this prefix is therefore safe — but only because CM.Exe independently selects the custom binary. Do not “simplify” by pointing CMStart.sh straight at bin/CarMaker.linux64; the GUI is required to load the TestRun, CMRosIF extension, and IPGControl hooks.)

Then, GUI / file checks:

1. Launch via bash `CMStart.sh`.
2. Confirm the **custom** binary is running, not the stock core:

```
ls -la ~/CM_Projects/FS_autonomous/bin/CarMaker.linux64 # must exist,
recent build
grep "CM.Exe" ~/CM_Projects/FS_autonomous/Data/Config/GUI # must show
CarMaker.linux64
```

3. Confirm CMRosIF present: **Extras** menu appears, status online.
4. Confirm CMNode alive: terminal prints `hellocm: I am Alive`.
5. **Disable unsupported GPU sensors** (4, 5, 6) in
Data/Vehicle/Examples_FS/FS_Autonomous → Sensor.4/5/6.Active = 0.
6. **Restore and lock the TestRun** Data/TestRun/FS_autonomous_TrackDrive (values per commit 6182620), then:

```
chmod 444 ~/CM_Projects/FS_autonomous/Data/TestRun/FS_autonomous_TrackD
rive
```

KNOWN ISSUE (Tcl load-order): If startup throws invalid command name `::CMRosIF::ROS_Launch`, the launch is called before the interface loads. Fix in `.CarMaker.tcl`: defer with `after idle`, order `Application connect` **before** `LoadTestRun` (full content in §6.5).

3.6 Launch Sequence (Strict Order)

Why: CarMaker must start first so CMNode exists; the bridge must start before Autoware so the subscriber is ready when Autoware begins publishing.

```
# 1) CarMaker + CMRosIF + CMNode
cd ~/CM_Projects/FS_autonomous
bash CMStart.sh

# 2) Bridge translator
source ~/ros2_ws/install/setup.bash
ros2 run bridge_node bridge_node

# 3) Autoware planning simulator
source ~/autoware/install/setup.bash
ros2 launch autoware_launch planning_simulator.launch.xml \
  map_path:=$HOME/autoware_map/sample-map-planning \
  vehicle_model:=sample_vehicle \
  sensor_model:=sample_sensor_kit

# 4) In RViz, in order:
#   2D Pose Estimate → place vehicle on a road → wait: Localization | Init
ialized
```

```
# 2D Goal Pose → set destination on road → wait: Routing | Set
# Accept Start → Auto
```

RViz initial pose — exact coordinates. “Place vehicle on a road” is only acceptable for a Mode-A isolation test, where the spawn point is arbitrary as long as it is on a lanelet. For **Mode B (closed loop)** the spawn pose is **not** free: Autoware’s initial base_link pose must coincide with CarMaker’s vehicle start position expressed in the shared origin (see §2.3). Until the Al Marjan map and a shared origin exist, there is **no valid fixed X,Y,Z,Yaw** to publish — any number would place Autoware’s ego somewhere CarMaker’s car is not.

Once the shared origin is defined, set the initial pose programmatically rather than by mouse, for repeatability:

```
ros2 topic pub --once /initialpose geometry_msgs/msg/PoseWithCovarianceStamped \
  "{header: {frame_id: 'map'}, pose: {pose: {position: {x: <X>, y: <Y>, z: <Z>},
orientation: {z: <sin(yaw/2)>, w: <cos(yaw/2)>}}}}"
```

<X>,<Y>,<Z>,<Yaw> come from CarMaker’s vehicle StartPos transformed into the Autoware map frame via the §2.3 static TF. **These values are a deliverable of the alignment work, not yet known.**

ROS 1 note: This project uses ROS 2 throughout. There is **no roscore** — ROS 2 has no central master. Discovery is handled by CycloneDDS.

Two operating modes — do not mix them: - **Mode A — Autoware alone (no CarMaker):** the simple_planning_simulator publishes vehicle status internally. Use to verify Autoware planning in isolation. Running CarMaker *and* the planning simulator together makes them fight over the vehicle interface and locks the **Auto** button. - **Mode B — Full integration:** CarMaker + bridge. The bridge must publish /vehicle/status/* back to Autoware (the return path). This is the real SP1 closed loop.

4. System Verification & Troubleshooting (Verify)

4.1 Unit Tests — Confirm the Bridge Is Active

```
# A) Is CMNode actually subscribing to control input?
ros2 topic info /carmaker/VehicleControl --verbose
# PASS: Subscription count: 1 FAIL: Subscription count: 0

# B) Is the bridge emitting correct values under command?
ros2 topic echo /carmaker/VehicleControl
# PASS: use_vc=True, non-zero gas/brake when Autoware commands motion

# C) Is Autoware publishing control commands?
ros2 topic echo /control/command/control_cmd
```

```
# PASS: non-zero longitudinal.acceleration / steering_tire_angle when engaged

# D) Rate check on the ZOH output
ros2 topic hz /carmaker/VehicleControl      # expect ~1000 Hz
```

Expected downstream signals when Autoware commands forward motion:

Signal	Expected
/control/command/control_cmd → longitudinal.acceleration	positive (e.g. 1.5 m/s ²)
/carmaker/VehicleControl → gas	e.g. 0.5
/carmaker/VehicleControl → brake	0.0
/carmaker/VehicleControl → steer_ang	~0.0 (straight)
IPGControl → VC.Gas	rising from 0
IPGControl → Car.v	increasing from 0

Reference acceptance result (Level 1): car drove autonomously on the Nishishinjuku sample map, Motion | Moving green, **8 km/h**.

4.2 Troubleshooting Matrix

#	Failure (Symptom)	Root Cause	Exact Fix
1	Subscription count: 0 on /carmaker/VehicleControl; CMStart terminal silent on sim start/stop; car ignores Autoware	src/User.cpp had no loader code, so libCMNode_ROS2_HelloCM.so was never dlopen'd; and bin/ was empty, so CMStart.sh ran the stock /opt/ipg binary with no CMRosIF	Add CMRosIF loader code to User.cpp; add -Wl, -E (export-dynamic) link flag; make the custom bin/CarMaker.linux64; point CMStart.sh at it. Verify: Subscription count: 1
2	CarMaker reports no valid license on Linux (License File field blank)	License No. 682015 was generated for Windows with hostname ali_saeed (underscore). Linux rejects underscores (RFC) → kernel hostname is alisaeeed. CarMaker validates hostname via uname(), which returns alisaeeed → mismatch. (Also: installing CarMaker 14.2.1 instead of 15.0 fails — the license is v15.)	Install CarMaker 15.0 (match the license version). Request a Linux license from IPG (support@ipg-automotive.com) with hostname alisaeeed, same MAC 60:e3:2b:68:6e:88, same sysid 306736293. Install to /opt/ipg/etc/licenses. (Technical fallback: LD_PRELOAD shim on uname() returning ali_saeed.)
3	SIM_ABORT on start; later sim won't run past Time 0.002	TestRun expected traffic cones + ego on route ObjId=3 not present on the flat track;	Restore Road.FName = FS_autonomous_TrackDrive.rd5, Routing.Type = Route,

#	Failure (Symptom)	Root Cause	Exact Fix
		deeper: the CarMaker GUI auto-overwrites Road.FName and routing lines every time the TestRun is opened, silently corrupting the file	StartPos.Type = Route, ObjId = 3; strip stray traffic. Then chmod 444 the TestRun so the GUI cannot rewrite it.
4	CMRosIF workspace won't compile	IPG CMake scripts have hardcoded version 14.1.1 (cmrosutils/CMakeLists.txt) and a missing \$ENV{CARMAKER_VER} value (carmaker_rlds_client/CMakeLists.txt)	Edit both files (14.1.1→15.0; add the \$ENV{} value). Set CARMAKER_VER=15.0 + IPGHOME=/opt/ipg. Wipe caches (rm -rf build install log). Rebuild → 7 packages compile.
5	Sim stalls initializing GPU sensors ; invalid command name "::oo::Obj14474"	Unsupported GPU camera/LiDAR processes (Sensors 4, 5, 6) crash on the Intel/NVIDIA hybrid GPU	In Data/Vehicle/Examples_FS/FS_Autonomous set Sensor.4/5/6.Active = 0 (via sed or editor).
6	Auto button locked , or Motion flickers green→red	CarMaker and the simple_planning_simulator running together fight over the vehicle interface; or no map loaded so Routing Unknown	Run Mode A or Mode B, never both. For isolation: close CarMaker, launch Autoware with a valid map_path, set Pose → Goal → Auto.

5. Current State & Next Action

Item	Status
Native Ubuntu 22.04 + NVIDIA/CUDA	Done
ROS 2 Humble + CycloneDDS	Done
Autoware Universe (482 pkgs)	Done
CarMaker 15.0 + license	Done
CMRosIF (7 pkgs) + custom binary	Done
bridge_node.py forward path	Done — Subscription count: 1
First autonomous drive (Level 1)	Done — 8 km/h
Return path (Level 2)	In implementation

Immediate next action — implement the return path in CMNode. Full design contract, DVA list, and quantified success criteria are in **§7 (Level 2 Specification)**. Summary: 1. Probe DVA names in IPGControl: confirm Car.v and the exact steering DVA (do not assume Car.Steer.Ang — verify against CM 15.0; §7.1). 2. In CMNode_ROS2_HelloCM.{cpp,hpp}: add a publisher job mirroring cm2ext, hold a strong reference, add matching teardown

(§7.2). 3. In `bridge_node.py`: subscribe to the new state topic; publish `VelocityReport` + `SteeringReport` + `ControlModeReport` to `/vehicle/status/*` (§7.3). 4. Verify against the §7.4 quantified thresholds, not a qualitative “it moves.” 5. Resolve the map-origin alignment (§2.3) before expecting the closed loop to track.

6. Appendix — Verified Command Record (from terminal history)

These are the actual commands run, reconstructed from `~/.bash_history`. They correct and supplement the narrative above. Where the development path involved trial-and-error, that is shown honestly so the team does not repeat dead ends.

6.1 CarMaker install (actual paths)

```
# Wrong version first (do NOT use – license is v15)
unzip CarMakerOffice-linux-14.1.2.zip -d CarMakerOffice-linux-14.1.2

# Correct version
unzip CarMakerOffice-linux-15.0.zip -d CarMakerOffice-linux-15.0
sudo ./ipg-install -ipghome /opt/ipg -usedir .
ls /opt/ipg/carmaker/linux64-15.0/bin/      # CM_Office-15.0, seedkey, cmlug
util, etc.
```

6.2 License — the full real sequence (with dead ends marked)

```
# License file actually used:
sudo cp ~/Downloads/Licenses.dat /opt/ipg/etc/licenses
sudo chmod 644 /opt/ipg/etc/licenses

# DEAD END 1 – gethostname() shim (wrong syscall; CarMaker uses uname())
echo '#include <unistd.h>'
#include <string.h>
int gethostname(char *name, size_t len){ strncpy(name,"ali_saeed",len); return 0; }' > /tmp/fake_host.c
gcc -shared -fPIC /tmp/fake_host.c -o /tmp/fake_host.so
export LD_PRELOAD=/tmp/fake_host.so      # did not work

# DEAD END 2 – forcing the hostname (underscore is RFC-invalid; reverts)
sudo hostnamectl set-hostname ali_saeed
sudo reboot                             # hostname won't hold the underscore

# WORKING FIX – correct the hostid, then obtain a proper Linux license from IPG
printf '\x25\x26\x4d\x12' | sudo tee /etc/hostid > /dev/null
hostid && xxd /etc/hostid
# License verified via:
/opt/ipg/carmaker/linux64-15.0/bin/cmplugutil -licinfo
```


Lesson for the team: Do not attempt the gethostname shim or hostnamectl underscore route. Request the Linux license from IPG with hostname alisaed from the start.

6.3 Autoware — install, map download, and the TensorRT blocker

```
# Source import (correct path)
vcs import src < repositories/autoware.repos
rosdep install -y --from-paths src --ignore-src --rosdistro $ROS_DISTRO
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release --parallel-workers 4

# Sample map (gdown) — and the map actually used:
mkdir -p ~/autoware_map && cd ~/autoware_map
wget https://github.com/tier4/AWSIM/releases/download/v1.1.0/nishishinjuku_autoware_map.zip
unzip nishishinjuku_autoware_map.zip
```

KNOWN ISSUE (TensorRT build failures): autoware_tensorrt_* packages failed. Resolution required pinning TensorRT to 8.6 and supplying CUDA/spconv/cumm paths explicitly:

```
TRT_VER=$(apt-cache madison libnvinfer-dev | awk '/8\.6/ {print $3}' | head -n1)
sudo apt-get install -y libnvinfer8=$TRT_VER libnvinfer-dev=$TRT_VER \
  libnvinfer-plugin8=$TRT_VER libnvinfer-plugin-dev=$TRT_VER \
  libnvonnxparsers8=$TRT_VER libnvonnxparsers-dev=$TRT_VER \
  libnvinfer-headers-dev=$TRT_VER libnvinfer-headers-plugin-dev=$TRT_VER
pip3 install spconv-cu120 cumm-cu120
CUMM_CMAKE=$(find ~/.local -name "cummConfig.cmake" | head -n1 | xargs dirname)
SPCONV_CMAKE=$(find ~/.local -name "spconvConfig.cmake" | head -n1 | xargs dirname)
CUDA_ROOT=$(dirname $(dirname $(which nvcc)))
colcon build --packages-up-to autoware_tensorrt_plugins autoware_tensorrt_classifier \
  --cmake-args -DCMAKE_BUILD_TYPE=Release \
  -Dcumm_DIR="${CUMM_CMAKE}" -Dspconv_DIR="${SPCONV_CMAKE}" \
  -DCUDA_TOOLKIT_ROOT_DIR="${CUDA_ROOT}" \
  -DTENSORRT_INCLUDE_DIR="/usr/include/x86_64-linux-gnu" \
  -DTENSORRT_LIBRARY="/usr/lib/x86_64-linux-gnu/libnvinfer.so"
```

6.4 CarMaker project setup (actual)

```
# Project came from FCM_Projects.zip; extracted from a cache path:
cp -r ~/.cache/.fr-*/FCM_Projects/FS_autonomous ~/CM_Projects/FS_autonomous

# CMStart.sh: Locate the GUI LAUNCHER (CM_Office-15.0) on an absolute path.
# NOTE: this is the GUI shell, NOT the simulation binary. The custom sim binary
# (bin/CarMaker.Linux64) is selected separately via Data/Config/GUI → CM.Exe
# (see §3.5).
```

```
# This sed does NOT bypass the custom binary.
sed -i 's|CM_Office-15.0|/opt/ipg/bin/CM_Office-15.0|' ~/CM_Projects/FS_autonomous/CMStart.sh

# ros_setup.bash pointed at the real workspace:
sed -i 's|ros/ros2_ws/install/setup.bash|/home/ali/ros2_ws/install/setup.bash|g' \
~/CM_Projects/FS_autonomous/ros_setup.bash
```

Cross-check (§3.5): after this, confirm Data/Config/GUI contains CM.Exe = CarMaker.linux64. If CM.Exe is blank, the GUI launches the stock simulation core with no CMRosIF loader and Subscription count returns to 0 — regardless of CMStart.sh.

6.5 .CarMaker.tcl — the working load-order fix (actual content)

```
Project::CheckProject
Application connect
after idle {
    LoadTestRun "FS_autonomous_TrackDrive"
    ::CMRosIF::ROS_Launch -mode launch
}
```

6.6 CMRosIF build — the actual CMake edits

```
export CARMAKER_VER=15.0
export IPGHOME=/opt/ipg
cd ~/CM_Projects/FS_autonomous/ros/ros2_ws

# cmrosutils — hardcoded version:
sed -i 's/set(CARMAKER_VER 14.1.1)/set(CARMAKER_VER 15.0)/' \
src/cmrosutils/CMakeLists.txt

# carmaker_rds_client — same hardcode AND a missing env value:
sed -i 's/set(CARMAKER_VER 14.1.1)/set(CARMAKER_VER 15.0)/' \
src/carmaker_rds_client/CMakeLists.txt
sed -i 's|set(CARMAKER_VER$|set(CARMAKER_VER "$ENV{CARMAKER_VER}")|' \
src/carmaker_rds_client/CMakeLists.txt

# Clean stale caches and rebuild:
rm -rf build/cmrosutils build/carmaker_rds_client
colcon build --symlink-install
```

6.7 GPU sensor disable (actual)

```
sed -i 's/^Sensor\.4\.Active = 1/Sensor.4.Active = 0/' \
~/CM_Projects/FS_autonomous/Data/Vehicle/Examples_FS/FS_Autonomous
sed -i 's/^Sensor\.5\.Active = 1/Sensor.5.Active = 0/' \
~/CM_Projects/FS_autonomous/Data/Vehicle/Examples_FS/FS_Autonomous
sed -i 's/^Sensor\.6\.Active = 1/Sensor.6.Active = 0/' \
~/CM_Projects/FS_autonomous/Data/Vehicle/Examples_FS/FS_Autonomous
```

6.8 The Subscription count: 0 blocker — what actually happened

The terminal record shows a long investigation before the real fix:

```
# Repeated checks (stayed at 0 through all of these):
ros2 topic info /carmaker/VehicleControl --verbose | grep "Subscription count"

# DEAD END — strong-ref edit to vc_sub_job_ at line ~851 (did NOT fix it):
sed -i 's/scheduler_.addJob(job);\n/scheduler_.addJob(job);\n      vc_sub_job_ = job;\n/' \
~/CM_Projects/FS_autonomous/ros/ros2_ws/src/hellocm_cmnode/src/CMNode_ROS2_HelloCM.cpp
colcon build --packages-select hellocm_cmnode      # rebuilt, still count 0
```

The actual fix (verified from git commit 6182620). The root cause had two layers, and the real fix was larger than first assumed — it was a **C→C++ migration of the entire CarMaker user-source layer**. The literal diffs follow.

Layer 1 — the user-source layer was C, but the CMRosIF loader is C++. The stock project shipped `src/User.c` (C), which loaded CMRosIF through a macro-gated helper `CMCppIFLoader_load("CMRosIF")` wrapped in `#if defined(WITH_CMCPPIF)`. That macro was never defined in this project's build, so **every loader call compiled to nothing** — the `.so` was never loaded. The fix renamed `User.c` → `User.cpp` and replaced the macro helper with an explicit C++ loader. The essential new code in `User.cpp`:

```
/* CMRosIF C++ interface Loader */
#include "cmrosutils/cmcppif.h"

static tSharedLib s_CMRosIF_lib = SHAREDLIB_NOT_LOADED;
static cmcppif::CarMakerCPPInterface* s_CMRosIF = nullptr;
static tInfos* s_CMRosIF_inf = nullptr;

int User_Init(void) {
    s_CMRosIF_inf = InfoNew();
    if (iRead2(nullptr, s_CMRosIF_inf, "Data/Config/CMRosIFParameters", "CMRosIF") < 0) {
        InfoDelete(s_CMRosIF_inf); s_CMRosIF_inf = nullptr;
        return 0; /* CMRosIFParameters absent → CMRosIF disabled */
    }
    if (iGetIntOpt(s_CMRosIF_inf, "Cfg.Mode", 0) == 0) return 0;

    const char* lib_path = iGetStrOpt(s_CMRosIF_inf, "Cfg.Lib.Path", "");
    s_CMRosIF_lib = CM_LoadLibrary(lib_path); /* dlopen the .so */
    auto creator = reinterpret_cast<cmcppif_creator_t>(
        CM_GetSymbol(s_CMRosIF_lib, "create")); /* find entry point */
    s_CMRosIF = creator(); /* instantiate node */
}
```

```

    s_CMROSIF->init(s_CMROSIF_inf);                                /* initialise
*/
    return 0;
}

```

Every CarMaker hook (User_In, User_DrivMan_Calc, User_VehicleControl_Calc, User_Calc, User_Out, User_TestRun_*, User_End) was rewired from the CMCppIFLoader_* macros to direct s_CMROSIF->... calls.

Layer 2 — the linker couldn't resolve CarMaker symbols. The loaded .so references CarMaker API symbols (Car, Vehicle, SimCore, ...) that live in the main binary. Without exporting them, dlopen fails to bind. Fix in src/Makefile:

```

# was: include ../linux64-14.1.1/include/MakeDefs.linux64
include /opt/ipg/carmaker/linux64-15.0/include/MakeDefs.linux64

# CMROSIF C++ interface header (provides cmcppif.h for the User.cpp loader)
CMROSIF_INC = /home/ali/CM_Projects/FS_autonomous/ros/ros2_ws/install/cmrosut
ils/include
INC_CFLAGS += -I$(CMROSIF_INC)

# Export all symbols so dlopen-loaded libCMNode_ROS2_HelloCM.so can resolve
# CarMaker API symbols (Car, Vehicle, SimCore, ...)
LD_FLAGS += -Wl,-E

# Link step switched from C compiler ($(CC)) to C++ compiler ($(CXX)):
$(APP_NAME): ...
    $(CXX) $(CXXFLAGS) $(LD_FLAGS) -o $@ $(OBS) $(LD_LIBS) ...

```

Layer 3 — a secondary rclcpp lifetime bug in CMNode. Once loading worked, the VehicleControl subscriber still needed a strong reference held, because rclcpp stores subscriptions as weak_ptr internally. The 2-line CMNode fix (CMNode_ROS2_HelloCM.cpp / .hpp):

```

// in userTestrunStartAtBegin(), after scheduler_.addJob(job):
vc_sub_job_ = job; // hold strong ref; rclcpp keeps subscriptions as weak_p
tr

// in userTestrunEnd(), with the other deleteJob calls:
vc_sub_job_.reset();

// in the header (CMNode_ROS2_HelloCM.hpp):
std::shared_ptr<CMJob::AbstractJob> vc_sub_job_;

```

Build-script support (ros/ros2_ws/build.sh) was also hardened to export IPGHOME=/opt/ipg and CARMAKER_VER=15.0 and pass --cmake-args -DCARMAKER_VER=15.0 so the value overrides any stale CMake cache.

Outcome: the custom bin/CarMaker.linux64 now dlopens libCMNode_ROS2_HelloCM.so at startup; Subscription count: 1 confirmed.

Why a stock /opt/ipg binary can never work: it contains the unmodified C user layer with `WITH_CMCPPIF` undefined, so it has no loader and no exported symbols. The project's own custom binary, built from this commit, is mandatory.

Reference: full diffs are in git commit 6182620 ("Fix CMRosIF loading: add loader to User.cpp + export-dynamic linker flag"), files `src/User.cpp`, `src/Makefile`, `ros/ros2_ws/build.sh`, and `ros/ros2_ws/src/hellocm_cmnode/src/CMNode_ROS2_HelloCM.{cpp,hpp}`.

6.9 TestRun fix — honest record (it was trial-and-error, not one clean edit)

The routing type was changed repeatedly before landing on a working config: Route → Road → Curvey → Lane, plus `ObjId 3 → 0`, deletion of stray routing lines, and clearing of CarMaker's own temp/cache files:

```
rm -rf /tmp/CM_*
rm -f ~/CM_Projects/FS_autonomous/Data/Vehicle/.tmp_alisaheed_ali.0
sed -i 's/^Traffic.N = .*/Traffic.N = 0/' \
    ~/CM_Projects/FS_autonomous/Data/TestRun/FS_autonomous_TrackDrive
# Final state then Locked read-only:
chmod 444 ~/CM_Projects/FS_autonomous/Data/TestRun/FS_autonomous_TrackDrive
```

The reason the file kept reverting (and why so many edits were needed) was the GUI auto-overwrite. Once `chmod 444` was applied, edits held.

6.10 Complete bridge_node.py (verified source)

This is the exact forward-path bridge as deployed. The return-path (Level 2) subscribers will be added to this same node.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from autoware_control_msgs.msg import Control
from vehiclecontrol_msgs.msg import VehicleControl

class BridgeNode(Node):
    def __init__(self):
        super().__init__("bridge_node")
        self.timer = self.create_timer(0.001, self.timer_callback) # ZOH at
1000 Hz
        self.acc = None
        self.steer = None
        self.Gas = 0.0
        self.Brake = 0.0
        self.steer_output = 0.0
        self.cmd_control = self.create_subscription(
            Control, "/control/command/control_cmd", self.control_callback, 1
0)
        self.publish_control = self.create_publisher(
```

```

        VehicleControl, "/carmaker/VehicleControl", 10)

def control_callback(self, msg: Control):
    # receives command from Autoware at 10 Hz
    self.steer = msg.lateral.steering_tire_angle
    self.acc = msg.longitudinal.acceleration

MAX_ACCEL = 3.0    # m/s2 → full gas
MAX_DECEL = 3.0    # m/s2 → full brake

def timer_callback(self):
    # republishes held values at 1000 Hz (ZOH); converts accel → gas/brake
    e
    msg = VehicleControl()
    if self.acc is not None:
        if self.acc >= 0.0:
            # clamp to [0,1]: a 3.1 m/s2 spike saturates to full gas, never drops the update
            self.Gas = min(max(self.acc / self.MAX_ACCEL, 0.0), 1.0)
            self.Brake = 0.0
        else:
            self.Gas = 0.0
            self.Brake = min(max(-self.acc / self.MAX_DECEL, 0.0), 1.0)
    if self.steer is not None:
        self.steer_output = self.steer
    msg.gas = self.Gas
    msg.brake = self.Brake
    msg.steer_ang = self.steer_output
    msg.steer_ang_vel = 0.0    # TODO: differentiate in validation phase
    msg.steer_ang_acc = 0.0    # TODO: differentiate in validation phase
    msg.selector_ctrl = 1
    msg.use_vc = True
    self.publish_control.publish(msg)
    self.get_logger().info(
        f"Gas: {self.Gas}, Brake: {self.Brake}, Steer: {self.steer_output}"),
        throttle_duration_sec=1.0)

def main(args=None):
    rclpy.init(args=args)
    node = BridgeNode()
    rclpy.spin(node)
    rclpy.shutdown()

```

Notes for the team: - MAX_ACCEL / MAX_DECEL are 3.0 m/s². Conversion uses true mathematical clamping $\min(\max(x, 0.0), 1.0)$, so an out-of-band command (e.g. 3.1 m/s²) **saturates to full gas/brake** rather than being discarded. This is the corrected logic; the earlier band-check version (which silently held the previous value on out-of-range input) must not be used. - steer_ang_vel and steer_ang_acc are stubbed at 0.0 and will

be computed by numerical differentiation of `steer_ang` during the validation phase. - The 0.0 initial values for `self.Gas/Brake/steer_output` mean the very first ticks before any Autoware message publish a safe neutral command.

6.11 Data/Config/CMRosIFParameters — REQUIRED runtime config (build-breaker if missing)

The C++ loader in `User_Init()` (§6.8) reads this file via `iRead2(..., "Data/Config/CMRosIFParameters", "CMRosIF")`. **If the file is missing, `Cfg.Mode` is 0, or `Cfg.Lib.Path` is empty, the loader returns early and CMRosIF never loads** — reproducing Subscription count: 0 with no error. It is a plain CarMaker InfoFile. The loader reads exactly two keys (`Cfg.Mode`, `Cfg.Lib.Path`); the file must contain at minimum:

```
#INFOFILE1.1 (UTF-8) - Do not remove this line!
CMRosIF.Cfg.Mode = 1
CMRosIF.Cfg.Lib.Path = <ABSOLUTE PATH to libCMNode_ROS2_HelloCM.so>
```

- **`Cfg.Mode = 1`** enables CMRosIF (0 disables it — the loader aborts cleanly).
- **`Cfg.Lib.Path`** is the absolute path to the compiled CMNode shared library produced by the 7-package colcon build (§3.4). It is built into the CMRosIF workspace install tree; **confirm the exact path on your build** before writing it:

```
find ~/CM_Projects/FS_autonomous/ros/ros2_ws/install -name "libCMNode_ROS2_HelloCM.so"
```

Typical value:

```
~/CM_Projects/FS_autonomous/ros/ros2_ws/install/hellocm_cmnode/lib/libCMNode_ROS2_HelloCM.so (use the absolute path the find returns, with $HOME expanded — InfoFiles do not expand ~).
```

The loader reads only these two keys, so the file is minimal. The InfoFile key prefix is `CMRosIF`. because the loader passes "CMRosIF" as the section argument to `iRead2`; the code then queries `Cfg.Mode` and `Cfg.Lib.Path` within that section.

6.12 ROS 2 package config — setup.py, setup.cfg, package.xml (REQUIRED to rebuild from scratch)

If the `bridge_node` repo (§3.3) is ever lost and the workspace is rebuilt from just `bridge_node.py`, `ros2 run bridge_node bridge_node` will fail without these three files. They define the package and the executable entry point.

Directory layout (ament_python package):

```
~/ros2_ws/src/bridge_node/
├── bridge_node/
│   ├── __init__.py          (empty file)
│   └── bridge_node.py       (the node, §6.10)
└── resource/
```

```
├── bridge_node          (empty marker file)
├── package.xml
├── setup.py
└── setup.cfg
```

package.xml:

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schemat
ypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>bridge_node</name>
  <version>0.0.1</version>
  <description>Autware ↔ IPG CarMaker control bridge (SP1)</description>
  <maintainer email="alialsaid248@gmail.com">Ali Alsaid</maintainer>
  <license>Proprietary</license>

  <depend>rclpy</depend>
  <depend>autoware_control_msgs</depend>
  <depend>vehiclecontrol_msgs</depend>
  <!-- Level 2 (return path) will add: -->
  <!-- <depend>autoware_vehicle_msgs</depend> -->

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

setup.py — the `console_scripts` entry point is what makes `ros2 run bridge_node` `bridge_node` work:

```
from setuptools import setup

package_name = 'bridge_node'

setup(
    name=package_name,
    version='0.0.1',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Ali Alsaid',
    maintainer_email='alialsaid248@gmail.com',
    description='Autware <-> IPG CarMaker control bridge (SP1)',
    license='Proprietary',
```



```

    entry_points={
        'console_scripts': [
            # 'ros2 run bridge_node bridge_node' → bridge_node.bridge_node:main
            'bridge_node = bridge_node.bridge_node:main',
        ],
    },
)

```

setup.cfg:

```

[develop]
script_dir=$base/lib/bridge_node
[install]
install_scripts=$base/lib/bridge_node

```

After creating these, `cd ~/ros2_ws && colcon build --packages-select bridge_node && source install/setup.bash` makes `ros2 run bridge_node bridge_node` resolve. The entry-point name on both sides of the `=` matters: left = the command, right = `<package>.<module>:<function>`.

7. Level 2 Specification — Return Path (IN IMPLEMENTATION)

Status: not yet built. The C++ publisher block below does **not exist in the codebase yet** — this section is the design contract for the work in progress, not a record of completed code. Do not treat the snippet as copy-paste-ready; it is the agreed shape of the implementation, to be written, compiled, and verified, then folded back into §6 once real.

7.1 DVA variables to probe

The return path needs CarMaker’s true physics state each tick. The minimum set, to be confirmed by name in IPGControl against the running sim before coding:

Autoware need	CarMaker DVA (candidate)	Unit	Confirm in IPGControl
Vehicle longitudinal speed	<code>Car.v</code>	m/s	required
Actual road-wheel steer angle	<code>Car.Steer.Ang</code> (<i>verify exact name; may be <code>Steer.WhlAng</code> or <code>Vhcl.Steer.Ang</code> in CM 15.0</i>)	rad	required
Position (for alignment check, §2.3)	<code>Car.Fr1.tx</code> , <code>Car.Fr1.ty</code>	m	optional, for TF validation

Autoware need	CarMaker DVA (candidate)	Unit	Confirm in IPGControl
Yaw (for alignment check)	Car.Yaw	rad	optional

Probe procedure: with the sim running, open IPGControl → add quantity → search the dictionary for Car.v and the steer candidates. The exact steering DVA name in CM 15.0 must be confirmed empirically; do not assume Car.Steer.Ang is correct until verified. This mirrors the Steer.WhlAng discovery from the earlier IPGControl telemetry work.

7.2 CMNode publisher (design contract — to be written)

Following the existing cm2ext / wheel-speeds publisher pattern (§6 references lines ~494, ~559–563, ~865–874, ~927). Shape:

```
// In CMNode_ROS2_HelloCM.hpp – declare the publisher job + state holders:
std::shared_ptr<CMJob::AbstractJob> vehicle_state_job_;
double state_v_;           // Car.v
double state_steer_;       // confirmed steering DVA

// In userTestrunStartAtBegin() – set up the publisher (mirror cm2ext):
//   typedef CMJob::RosPublisher<vehiclestate_msgs::msg::VehicleState> vstate
//   _t;
//   auto job = std::make_shared<vstate_t>(nhp_, "vehicle_state");
//   job->registerCallback(&CMNodeHelloCM::vehicleStateFillMsg, this);
//   scheduler_.addJob(job);
//   vehicle_state_job_ = job;           // strong ref (same weak_ptr lesson a
//   s VehicleControl)

// Fill callback – read DVAs each tick, populate the message:
//   void CMNodeHelloCM::vehicleStateFillMsg(msg& m) {
//       m.v      = Car.v;               // or the bound DVA variable
//       m.steer = <confirmed steer DVA>;
//   }

// In userTestrunEnd() – matching teardown (REQUIRED or CarMaker crashes on s
// top):
//   scheduler_.deleteJob("vehicle_state");
//   vehicle_state_job_.reset();
```

Two non-negotiables learned the hard way (§6.8): (1) hold a **strong reference** to the job — rclcpp keeps subscriptions/publishers as weak_ptr. (2) Add the matching deleteJob + .reset() in teardown, or the sim crashes on stop.

7.3 Bridge return half (Python — Ali owns this)

Add to bridge_node.py: subscribe to the new /carmaker/vehicle_state topic; republish into the topics Autoware’s localizer/controller consume:

From CarMaker	To Autoware topic	Message
state_v_	/vehicle/status/velocity_status	autoware_vehicle_msgs/msg/VelocityReport
state_steer_	/vehicle/status/steering_status	autoware_vehicle_msgs/msg/SteeringReport
(constant AUTONOMOUS)	/vehicle/status/control_mode	autoware_vehicle_msgs/msg/ControlModeReport

7.4 Level 2 Success Criteria (quantified)

Level 1 success was qualitative (“car moves at 8 km/h”). Level 2 must be **measured**. Define PASS as **all** of:

Metric	PASS threshold	How measured
Velocity feedback present	/vehicle/status/velocity_status publishing at ≥ 50 Hz, non-zero under throttle	ros2 topic hz + echo
Lateral tracking error	vehicle stays within ± 0.5 m of the planned trajectory centerline over a 200 m run	compare base_link vs planned path in RViz / rosbag
Longitudinal speed tracking	actual Car.v tracks Autoware target speed within ± 1.0 km/h in steady state	overlay target vs Car.v in IPGControl
Origin alignment (§2.3)	CarMaker pose (via static TF) matches Autoware base_link within ± 0.2 m at start	tf2_echo + CarMaker Car.Fr1.tx/ty
Loop stability	200 m completed with no oscillation/divergence (steering does not hunt)	visual + steering-angle trace

These thresholds are **proposed starting values** for the doctor’s review. The lateral ± 0.5 m and speed ± 1.0 km/h margins should be confirmed against RAKTA’s actual acceptance requirements for the Al Hamra anomaly study before they become binding.

Authoritative source paths, versions, and identifiers in this document are exact and verified against the full development record: chat history, terminal ~/.bash_history, Claude Code session logs, and the literal git diff of commit 6182620. Sections 2.3, 3.3, 3.6 and 7 contain items explicitly marked as OPEN / FILL-IN / TO-BE-WRITTEN — these are honest gaps requiring author or RAKTA input, not omissions. Update §5 and §7 as the return path (Level 2) closes.